

UniCaronas

Sistema de Caronas Universitárias

Relatório de Resultados — Sprint 1

Disciplina: Prática em Programação Web — 3ESCN

Equipe: Rafael Machado, Ariane Archanjo, Pedro Kafka, Matheus Sizanoski

1. Resumo Executivo

A Sprint 1 do projeto UniCaronas foi concluída com sucesso no período de 18 a 25 de março de 2026. Todas as três histórias de usuário planejadas foram entregues dentro do prazo, totalizando 11 Story Points. O sistema passou a contar com fluxo completo de autenticação: cadastro, login e visualização de perfil funcionando de ponta a ponta no navegador.

Métrica	Valor
Story Points planejados	11 SP
Story Points entregues	11 SP (100%)
Histórias concluídas	3 de 3 (US01, US02, US03)
Velocidade	5,5 SP / semana
Impedimentos resolvidos	2 de 2
Status geral	☒ Sprint concluída

2. Histórias de Usuário Entregues

US01 — Criar conta no sistema (5 pontos)

Como estudante, quero criar uma conta com meu e-mail da universidade para entrar no sistema com segurança.

Critério de Aceitação	Status
Campos nome, e-mail, matrícula e senha obrigatórios	☒ Implementado
Senha com mínimo de 8 caracteres	☒ Implementado
E-mail com domínio institucional configurável via .env	☒ Implementado
Aviso em caso de e-mail ou matrícula duplicados	☒ Implementado
Redirecionamento para login após cadastro	☒ Implementado
Senha armazenada com bcrypt (hash)	☒ Implementado

US02 — Entrar no sistema — Login (3 pontos)

Como usuário cadastrado, quero entrar com meu e-mail e senha para acessar o sistema.

Critério de Aceitação	Status
Login via e-mail + senha com resposta JWT	☒ Implementado
Token com validade de 24h	☒ Implementado
Mensagem de erro em credenciais inválidas	☒ Implementado
Páginas internas protegidas para usuários não autenticados	☒ Implementado
Token armazenado no localStorage	☒ Implementado

US03 — Ver e editar o perfil (3 pontos)

Como usuário logado, quero ver e editar meu perfil para que outros me conheçam antes da carona.

Critério de Aceitação	Status
Exibe nome, e-mail, curso e avaliação média	☒ Implementado
Exibe estatísticas de caronas como motorista e passageiro	☒ Implementado
Permite editar telefone e dados pessoais	☒ Implementado
Redireciona para login se não autenticado	☒ Implementado
Exibe avaliações recebidas (seção)	☒ Implementado

3. Telas Implementadas

Tela de Login (login.html)

Layout em duas colunas: painel visual à esquerda com headline e estatísticas, formulário à direita com campos de e-mail institucional e senha. Inclui link para cadastro e proteção de rota (redireciona para dashboard se já logado).

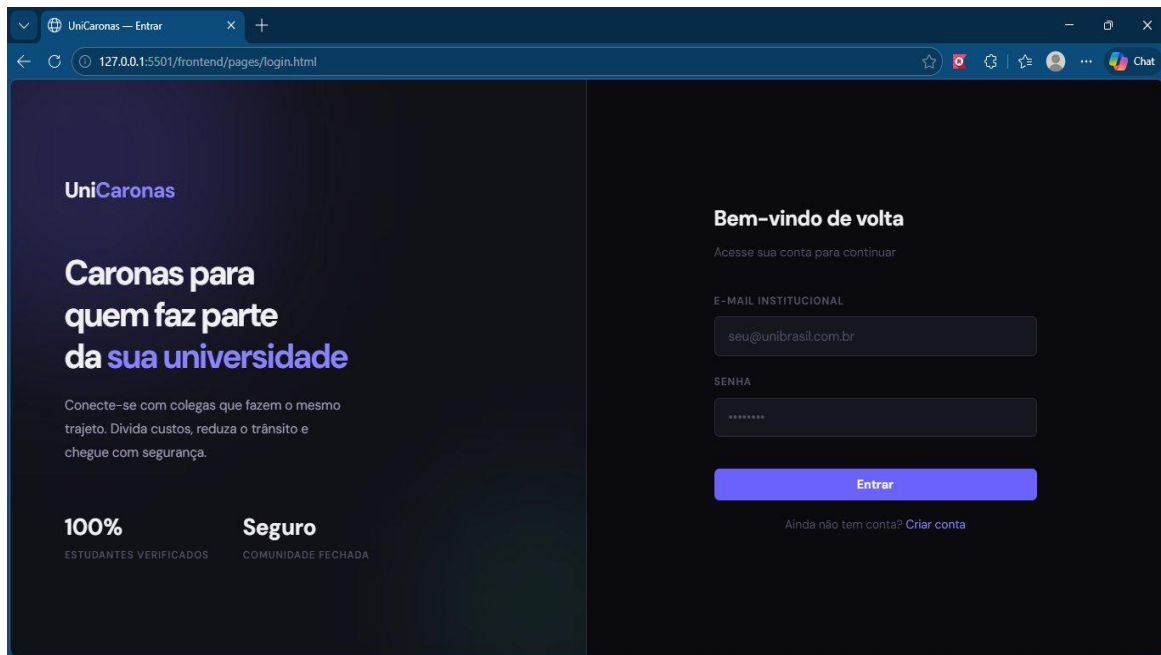


Figura 1 — Tela de Login

Tela de Cadastro (cadastro.html)

Formulário centralizado com campos: nome completo, matrícula, e-mail institucional, curso, telefone e senha. Validação client-side antes de enviar ao backend. Após cadastro com sucesso, redireciona para login.

Figura 2 — Tela de Cadastro

Dashboard (dashboard.html)

Painel principal com saudação personalizada por horário, três cards de estatísticas (caronas disponíveis, minhas caronas, minha avaliação), atalhos de ação rápida e listagem das caronas disponíveis. Navegação completa via navbar.

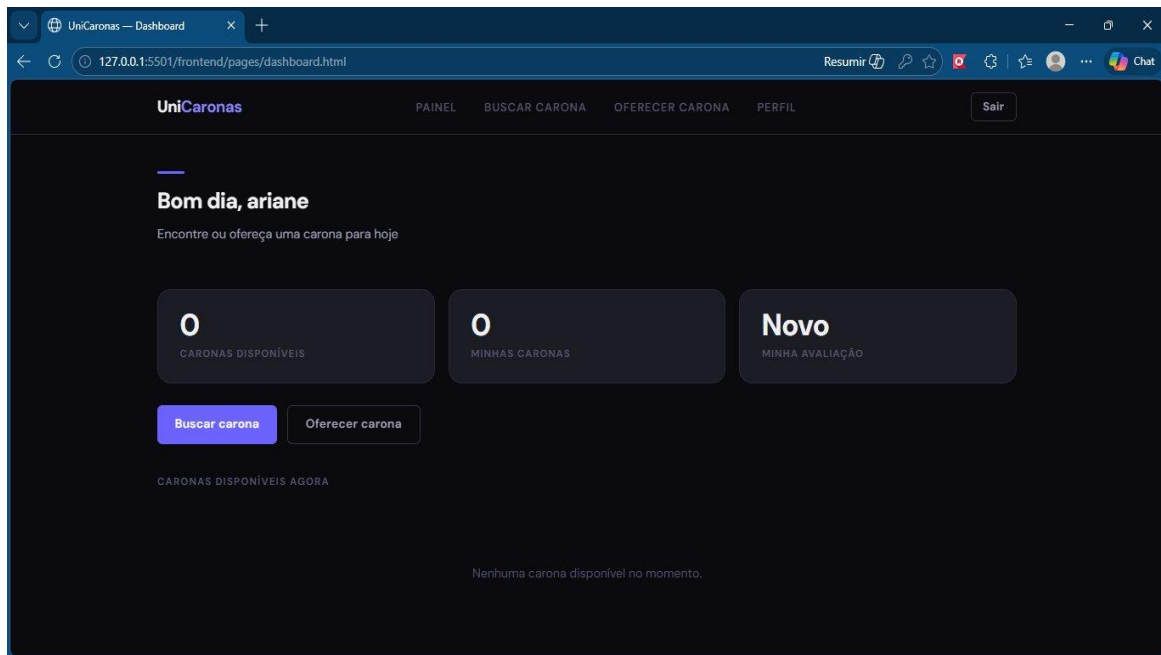


Figura 3 — Dashboard

Tela de Perfil (perfil.html)

Exibe avatar com iniciais geradas automaticamente, nome completo, curso, e-mail e data de criação da conta. Três cards de estatísticas (caronas como motorista, como passageiro, avaliação média com estrelas). Seção de avaliações recebidas e botão de edição de perfil.

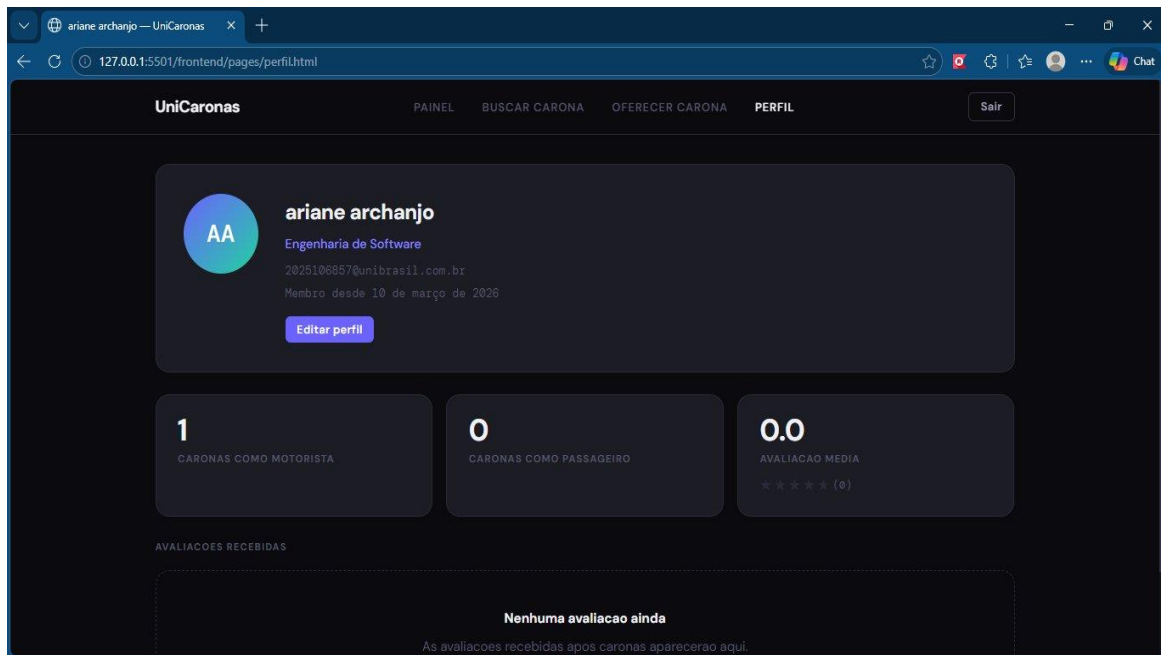


Figura 4 — Tela de Perfil

4. Principais Componentes de Código

4.1 Backend — Estrutura Geral

O backend foi construído com Node.js + Express seguindo padrão MVC. O arquivo `server.js` centraliza a configuração do servidor, middlewares globais e registro de rotas.

Arquivo	Responsabilidade
<code>server.js</code>	Ponto de entrada — configura Express, CORS, Helmet e rotas
<code>config/database.js</code>	Pool de conexões PostgreSQL com pg
<code>middleware/auth.js</code>	Validação do token JWT em rotas protegidas
<code>middleware/errorHandler.js</code>	Tratamento global de erros (status + mensagem)
<code>controllers/usuariosController.js</code>	Lógica de cadastro, login e atualização de perfil
<code>routes/usuarios.js</code>	Define endpoints: POST /, POST /login, GET /:id, PATCH /perfil

4.2 Autenticação JWT

O fluxo de autenticação é stateless. No login, o backend verifica as credenciais com `bcrypt`, gera um token JWT com validade de 24h assinado pela variável `JWT_SECRET`, e

retorna ao frontend. O middleware auth.js valida o header Authorization: Bearer <token> em todas as rotas protegidas.

Etapa	Implementação
Hash de senha	bcrypt.hash(senha, 10) no cadastro
Verificação	bcrypt.compare(senha, hash) no login
Geração do token	jwt.sign({ id, email, nome }, JWT_SECRET, { expiresIn: '24h' })
Validação	jwt.verify(token, JWT_SECRET) no middleware
Armazenamento client	localStorage (unicaronas_token + unicaronas_user)

4.3 Banco de Dados — Tabela usuarios

A tabela usuarios foi criada com validações a nível de banco: UNIQUE em email e matricula, DEFAULT para campos opcionais, e índices para consultas frequentes.

Campo	Tipo	Restrição
id	SERIAL	PK — auto-incremento
nome	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	UNIQUE, NOT NULL
matricula	VARCHAR(20)	UNIQUE, NOT NULL
senha_hash	VARCHAR(255)	NOT NULL
telefone	VARCHAR(20)	Opcional
curso	VARCHAR(100)	Opcional
avaliacao_media	DECIMAL(2,1)	DEFAULT 0.0
ativo	BOOLEAN	DEFAULT true
criado_em	TIMESTAMP	DEFAULT NOW()

4.4 Frontend — Módulo api.js

O módulo api.js centraliza toda comunicação com o backend. Utiliza fetch() com headers automáticos de autenticação, trata erros de forma padronizada e expõe funções utilitárias globais para as páginas HTML.

Função	Descrição
api.cadastrar(dados)	POST /api/usuarios — registra novo usuário

Função	Descrição
api.login(dados)	POST /api/usuarios/login — autentica e retorna token
api.perfil(id)	GET /api/usuarios/:id — busca dados do perfil
api.atualizarPerfil(dados)	PATCH /api/usuarios/perfil — edita perfil do usuário logado
protegerRota()	Redireciona para login se não autenticado
formatarData(iso)	Formata timestamp em data/hora pt-BR
formatarMoeda(v)	Formata número como R\$ BRL
iniciais(nome)	Gera sigla de 2 letras para o avatar

5. Impedimentos e Resoluções

Impedimento	Impacto	Resolução
Configuração do PostgreSQL no Windows	Bloqueou setup inicial do banco	Resolvido com WSL2
CORS bloqueando requisições do frontend	Frontend não conseguia chamar a API	Adicionado middleware cors() no Express

6. Retrospectiva da Sprint

O que funcionou bem

- Comunicação diária entre os membros da equipe — dúvidas resolvidas rapidamente
- Definição prévia do contrato de API (formato JSON esperado) evitou retrabalho entre frontend e backend
- 100% das tarefas planejadas foram entregues dentro do prazo da Sprint
- Fluxo completo de cadastro → login → perfil funcionando de ponta a ponta

O que pode melhorar

- Documentar endpoints à medida que são criados (não deixar para o final)
- Preparar ambiente de desenvolvimento (PostgreSQL, Node) antes do início da Sprint

7. Tecnologias Utilizadas

Camada	Tecnologia	Versão / Detalhe
Frontend	HTML5 + CSS3 + JavaScript	Sem frameworks — Vanilla JS
Backend	Node.js + Express	Express 4.x
Banco de Dados	PostgreSQL	v14+
Autenticação	JWT (jsonwebtoken)	v9.0
Hash de senha	bcrypt	v5.1
ORM/Driver	pg (node-postgres)	v8.x — pool de conexões
Segurança HTTP	Helmet + CORS	Headers seguros + CORS configurável
Dev server	Nodemon	Hot-reload em desenvolvimento
Testes	Jest + Supertest	Configurado para sprints futuras